

Chapter 11. Deployments and ReplicaSets

A major strength of Kubernetes lies in its ability to scale applications seamlessly and manage replication with ease. To enable these capabilities, Kubernetes provides two primitives: Deployments and ReplicaSets.

In this chapter, you'll learn how to create a Deployment, which leverages a ReplicaSet behind the scenes to manage and scale a group of identical Pods. Deployments also make it easy to roll out updates to your application and, if needed, roll back to a previous version—all with minimal downtime and maximum control.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Understand application deployments and how to perform rolling update and rollbacks
 - Understand the primitives used to create robust, self-healing, application deployments
-

Working with Deployments

The primitive for running an application in a container is the Pod. Using a single instance of a Pod to operate an application has its flaws—it represents a single point of failure because all traffic targeting the application is funneled to this Pod. This behavior is specifically problematic when the load increases due to higher demand (e.g., during peak shopping season for an ecommerce application or when an increasing number of microservices communicate with a centralized microservice functionality, like an authentication provider).

Another important aspect of running an application in a Pod is failure tolerance. The scheduler cluster component will not reschedule a Pod in the case of a node failure, which can lead to a system outage for end users. In

this chapter, we'll look at the Kubernetes mechanics that support application scalability and failure tolerance.

A *ReplicaSet* is a Kubernetes API resource that controls multiple, identical instances of a Pod running the application, called *replicas*. It has the capability of scaling the number of replicas up or down on demand.

A *Deployment* abstracts the functionality of ReplicaSet and manages it internally. In practice, this means you do not have to create, modify, or delete ReplicaSet objects yourself. The Deployment keeps a history of application versions and can delegate toward the ReplicaSet to roll back to an older version to counteract a blocking or potentially costly production issue. Furthermore, it offers the capability of scaling the number of replicas.

[Figure 11-1](#) illustrates the relationship between a Deployment, a ReplicaSet, and its controlled replicas.

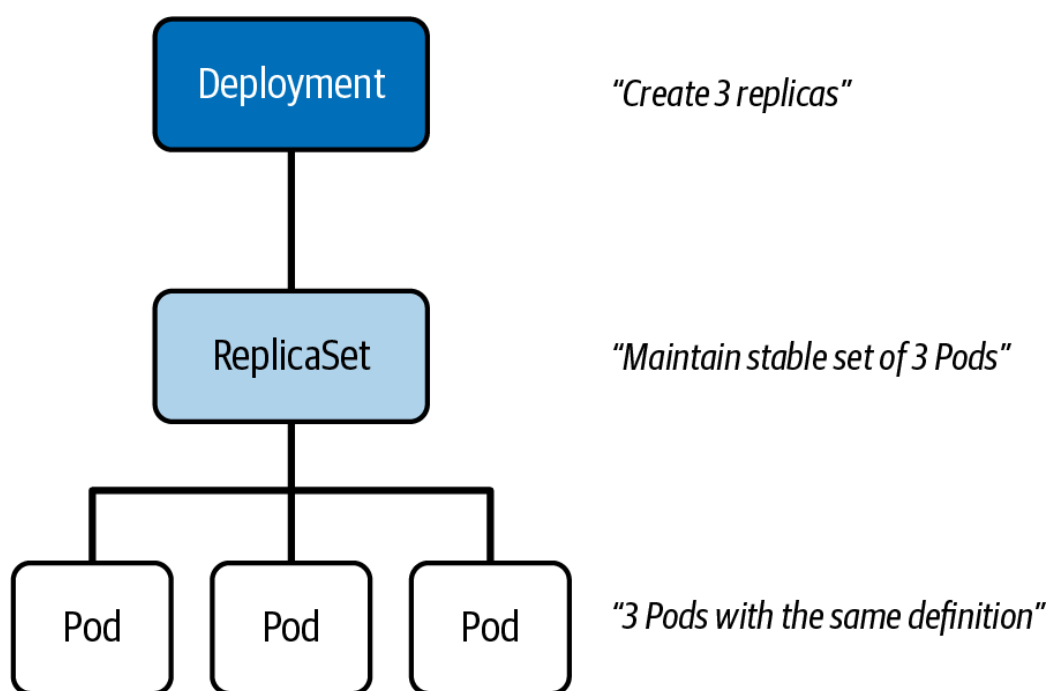


Figure 11-1. Relationship between a Deployment and a ReplicaSet

The following sections explain how to manage Deployments, including scaling and rollout features.

Creating Deployments

You can create a Deployment using the imperative command `create deployment`. The command offers a range of options, some of which are mandatory. At a minimum, you need to provide the name of the Deployment and the container image. The Deployment passes this information to the ReplicaSet, which uses it to manage the replicas. The de-

fault number of replicas created is one; however, you can define a higher number of replicas using the option `--replicas`.

Let's observe the command in action. The following command creates the Deployment named `app-cache`, which runs the object cache [memcached](#) inside the container on four replicas:

```
$ kubectl create deployment app-cache --image=memcached:1.6.8 --replicas=4
deployment.apps/app-cache created
```

The mapping between the Deployment and the replicas it controls happens through label selection. When you run the imperative command, `kubectl` sets up the mapping for you. [Example 11-1](#) shows the label selection in the YAML manifest. This YAML manifest can be used to create a Deployment declaratively or by inspecting the live object created by the previous imperative command.

Example 11-1. A YAML manifest for a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-cache
  labels:
    app: app-cache
spec:
  replicas: 4
  selector:
    matchLabels:
      app: app-cache
  template:
    metadata:
      labels:
        app: app-cache
    spec:
      containers:
        - name: memcached
          image: memcached:1.6.8
```

When created by the imperative command, `app` is the label key the Deployment uses by default. You can find this key in three different places in the YAML output:

- `metadata.labels`
- `spec.selector.matchLabels`

- `spec.template.metadata.labels`

For label selection to work properly, the assignment of `spec.selector.matchLabels` and `spec.template.metadata` needs to match, as shown in [Figure 11-2](#). Object creation will fail with an appropriate error message if the values of both assignments do not match.

The values of `metadata.labels` is irrelevant for mapping the Deployment to the Pod template. As you can see in the figure, the label assignment to `metadata.labels` has been changed deliberately to `deploy: app-cache` to underline that it is not important for the Deployment to Pod template selection.

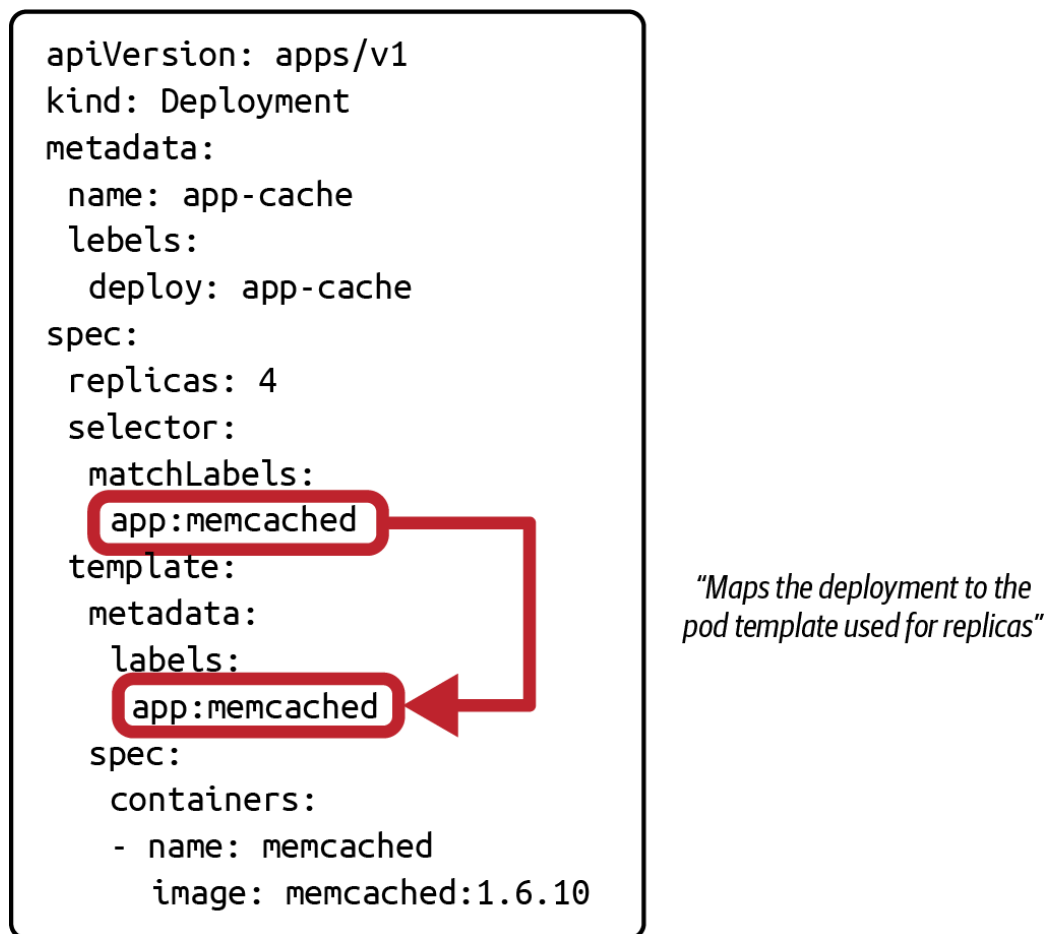


Figure 11-2. Deployment label selection

Listing Deployments, ReplicaSets, and Their Pods

You can inspect a Deployment after its creation by using the `get deployments` command. The output of the command renders the important details of its replicas, as shown here:

```
$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
app-cache    4/4      4             4           125m
```

The column titles relevant to the replicas controlled by the Deployment are shown in [Table 11-1](#).

Table 11-1. Runtime replica information when listing deployments

Column title	Description
READY	Lists the number of replicas available to end users in the format of <ready>/<desired>. The number of desired replicas corresponds to the value of <code>spec.replicas</code> .
UP-TO-DATE	Lists the number of replicas that have been updated to achieve the desired state.
AVAILABLE	Lists the number of replicas available to end users.

You can identify the Pods controlled by the Deployment by their naming prefix. In the case of the previously created Deployment, the names for ReplicaSet and its Pods start with `app-cache-`. The hash following the prefix is autogenerated and appended to the name upon creation:

```
$ kubectl get replicaset,pods
```

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-cache-596bc5586d	4	4	4	6h5m

NAME	READY	STATUS	RESTARTS	AGE
app-cache-596bc5586d-84dkv	1/1	Running	0	6h5m
app-cache-596bc5586d-8bzfs	1/1	Running	0	6h5m
app-cache-596bc5586d-rc257	1/1	Running	0	6h5m
app-cache-596bc5586d-tvm4d	1/1	Running	0	6h5m

Rendering Deployment Details

You can render the details of a Deployment. Those details include the label selection criteria, which can be extremely valuable when troubleshooting a misconfigured Deployment. The following output provides the full details:

```
$ kubectl describe deployment app-cache
```

```
Name: app-cache
Namespace: default
CreationTimestamp: Sat, 07 Aug 2021 09:44:18 -0600
Labels: app=app-cache
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=app-cache
Replicas: 4 desired | 4 updated | 4 total | 4 available |
```

```

0 unavailable
StrategyType:      RollingUpdate
MinReadySeconds:   0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=app-cache
  Containers:
    memcached:
      Image:      memcached:1.6.10
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
Conditions:
  Type           Status  Reason
  ----           -
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets: <none>
NewReplicaSet:  app-cache-596bc5586d (4/4 replicas created)
Events:         <none>

```

You might have noticed that the output contains a reference to a ReplicaSet. The purpose of a ReplicaSet is to *replicate* a set of identical Pods. You do not need to deeply understand the core functionality of a ReplicaSet for the exam. Just be aware that the Deployment automatically creates the ReplicaSet and uses the Deployment's name as a prefix for the ReplicaSet, similar to the Pods it controls. In the case of the previous Deployment named `app-cache`, the name of the ReplicaSet is `app-cache-596bc5586d`.

Replica Replacement

The ReplicaSet in Kubernetes ensures that a specified number of replicas are running at all times. If a Pod fails or is deleted, Kubernetes automatically creates a replacement Pod to maintain the desired replica count. This behavior is one of Kubernetes' key *self-healing capabilities*.

You can easily observe this in action by manually deleting one of the Pods managed by the ReplicaSet. Let's assume the ReplicaSet and Pod objects shown here:

```
$ kubectl get replicaset,pods
```

NAME	DESIRED	CURRENT	READY	AGE
------	---------	---------	-------	-----

replicaset.apps/app-cache-596bc5586d	4	4	4	6h47m
--------------------------------------	---	---	---	-------

NAME	READY	STATUS	RESTARTS	AGE
pod/app-cache-596bc5586d-84dkv	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-8bzfs	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-rc257	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-tvm4d	1/1	Running	0	6h47m

You can delete any of the Pods controlled by the ReplicaSet, i.e., the Pod named `app-cache-596bc5586d-rc257`:

```
$ kubectl delete pod app-cache-596bc5586d-rc257
pod "app-cache-596bc5586d-rc257" deleted
```

Shortly after deleting the Pod manually, a new one will be created by the ReplicaSet. You can easily identify the newly-created Pod by its `AGE` value. In the following output, the Pod is only five seconds old:

```
$ kubectl get replicaset,pods
```

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-cache-596bc5586d	4	4	4	6h47m

NAME	READY	STATUS	RESTARTS	AGE
pod/app-cache-596bc5586d-84dkv	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-8bzfs	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-lwflz	1/1	Running	0	5s
pod/app-cache-596bc5586d-tvm4d	1/1	Running	0	6h47m

Similar to the Deployment, other workload primitives like the [StatefulSet](#) and the [DaemonSet](#) manage a ReplicaSet to control a set of replicas.

For the StatefulSet, the replica replacement behavior is the same as for the Deployment. If a Pod fails as part of a DaemonSet, the control plane node ensures that the Pod replacement is run on the same node it was scheduled on before. The primitives StatefulSet and DaemonSet are out-of-scope for the exam; however, later chapters may mention them again.

Deleting a Deployment

A Deployment takes full charge of the creation and deletion of the objects it controls: ReplicaSets and Pods. When you delete a Deployment, the cor-

responding objects are deleted as well. Say you are dealing with the following set of objects shown in the output:

```
$ kubectl get deployments,replicasets,pods
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/app-cache	4/4	4	4	6h47m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-cache-596bc5586d	4	4	4	6h47m

NAME	READY	STATUS	RESTARTS	AGE
pod/app-cache-596bc5586d-84dkv	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-8bzfs	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-rc257	1/1	Running	0	6h47m
pod/app-cache-596bc5586d-tvm4d	1/1	Running	0	6h47m

Run the `delete deployment` command for a cascading deletion of its managed objects:

```
$ kubectl delete deployment app-cache
```

```
deployment.apps "app-cache" deleted
```

```
$ kubectl get deployments,replicasets,pods
```

```
No resources found in default namespace.
```

Performing Rolling Updates and Rollbacks

A Deployment fully abstracts rollout and rollback capabilities by delegating this responsibility to the ReplicaSet(s) it manages. Once a user changes the definition of the Pod template in a Deployment, Kubernetes will create a new ReplicaSet that applies the changes to the replicas it controls and then shut down the previous ReplicaSet. In this section, we'll see both scenarios: deploying a new version of an application and reverting to an old version of an application.

Updating a Deployment's Pod Template

You can choose from a range of options to update the definition of replicas controlled by a Deployment. Any of those options is valid, but they vary in ease of use and operational environment.

Applying changes declaratively

In real-world projects, you should check your manifest files into version control. Changes to the definition would then be made by directly editing the file. The `kubectl apply` command can update a live object by pointing to the changed manifest:

```
$ kubectl apply -f deployment.yaml
```

Editing the object from the default editor

The `kubectl edit` command lets you change the Pod template interactively by modifying the live object's manifest in an editor. To edit the Deployment live object named `web-server`, use the following command:

```
$ kubectl edit deployment web-server
```

Updating the container image imperatively

The imperative `kubectl set image` command changes only the container image assigned to a Pod template by selecting the name of the container. For example, you could use this command to assign the image `nginx:1.25.2` to the container named `nginx` in the Deployment `web-server`:

```
$ kubectl set image deployment web-server nginx=nginx:1.25.2
```

Replacing an existing object

The `kubectl replace` command lets you replace the existing Deployment with a new definition that contains your change to the manifest. The optional `--force` flag first deletes the existing object and then creates it from scratch. The following command assumes that you changed the container image assignment in *deployment.yaml*:

```
$ kubectl replace -f deployment.yaml
```

Updating fields of an object using a JSON patch

The command `kubectl patch` requires you to provide the merges as a patch to update a Deployment. The following command shows the operation in action. Here, you are sending the changes to be made in the form of a JSON structure:

```
$ kubectl patch deployment web-server -p '{"spec":{"template":{"spec":{"containers":[{"name":"nginx","image":"nginx:1.25.2"}]}}}}'
```

Rolling Out a New Revision

The Deployment primitive employs *rolling update* as the default deployment strategy, also referred to as *ramped deployment*. It’s called “ramped” because the Deployment gradually transitions replicas from the old version to a new version in batches. The Deployment automatically creates a new ReplicaSet for the desired change after the user updates the Pod template.

[Figure 11-3](#) shows a snapshot in time during the rollout process.

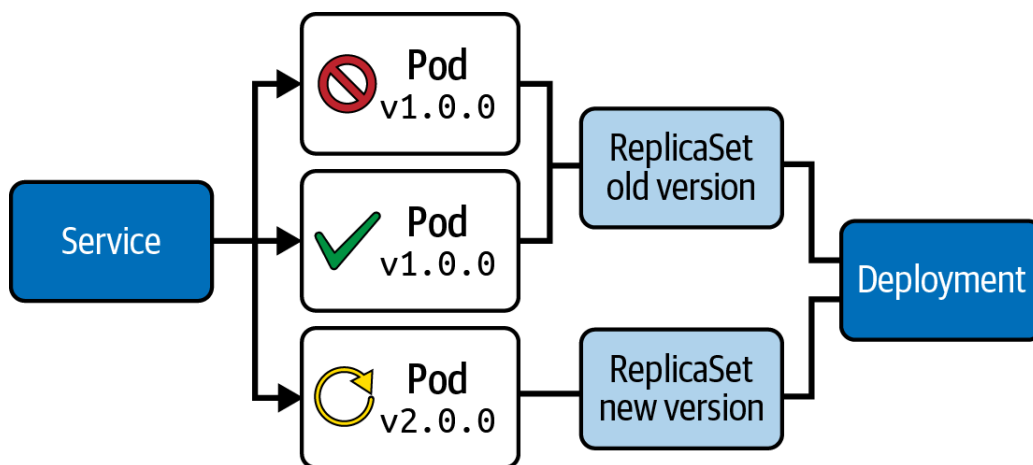


Figure 11-3. The rolling update strategy

In this scenario, the user initiated an update of the application version from 1.0.0 to 2.0.0. As a result, the Deployment creates a new ReplicaSet and starts up Pods running the new application version while at the same time scaling down the old version. The Service routes network traffic to either the old or new version of the application. The runtime behavior of the deployment strategy can be further customized. Refer to the [documentation](#) to see the configuration options.

Say you want to upgrade the version of memcached from 1.6.8 to 1.6.10 to benefit from the latest features and bug fixes. All you need to do is change

the desired state of the object by updating the Pod template. The command `set image` offers a quick, convenient way to change the image of a Deployment, as shown in the following command:

```
$ kubectl set image deployment app-cache memcached=memcached:1.6.10
deployment.apps/app-cache image updated
```

You can check the current status of a rollout that's in progress using the command `rollout status`. The output indicates the number of replicas that have already been updated since emitting the command:

```
$ kubectl rollout status deployment app-cache
Waiting for rollout to finish: 2 out of 4 new replicas have been updated
deployment "app-cache" successfully rolled out
```

Kubernetes keeps track of the changes you make to a Deployment over time in the rollout history. Every change is represented by a *revision*. When changing the Pod template of a Deployment—for example, by updating the image—the Deployment triggers the creation of a new ReplicaSet. The Deployment will gradually perform the migrating by decreasing the replica count of the old ReplicaSet and increasing the count on the new ReplicaSet. You can check the rollout history by running the following command. You will see two revisions listed:

```
$ kubectl rollout history deployment app-cache
deployment.apps/app-cache
REVISION    CHANGE-CAUSE
1           <none>
2           <none>
```

The first revision was recorded for the original state of the Deployment when you created the object. The second revision was added for changing the image tag.

CONFIGURABLE REVISION HISTORY

By default, a Deployment persists for a maximum of 10 revisions in its history. You can change the limit by assigning a different value to `spec.revisionHistoryLimit`.

To get a more detailed view of the revision, run the following command.

You can see that the image uses the value `memcached:1.6.10`:

```
$ kubectl rollout history deployments app-cache --revision=2
deployment.apps/app-cache with revision #2
Pod Template:
  Labels:      app=app-cache
              pod-template-hash=596bc5586d
  Containers:
    memcached:
      Image:      memcached:1.6.10
      Port:      <none>
      Host Port:  <none>
      Environment:  <none>
      Mounts:      <none>
  Volumes:      <none>
```

The rolling update strategy ensures that the application is always available to end users. This approach implies that two versions of the same application are available during the update process. As an application developer, you have to be aware that convenience doesn't come without potential side effects. If you happen to introduce a breaking change to the public API of your application, you might temporarily break consumers, as they could hit revision 1 or 2 of the application.

You can change the default deployment strategy by providing a different value to the attribute `spec.strategy.type`; however, consider the trade-offs. The value Recreate kills all Pods first, then creates new Pods with the latest revision, causing potential downtime for consumers.

Adding a Change Cause for a Revision

The rollout history renders the column `CHANGE-CAUSE`. You can populate the information for a revision to document *why* you introduced a new change or *which* `kubectl` command you use to make the change.

By default, changing the Pod template does not automatically record a change cause. To add a change cause to the current revision, add an annotation with the reserved key `kubernetes.io/change-cause` to the Deployment object. The following imperative `annotate` command assigns the change cause "Image updated to 1.6.10":

```
$ kubectl annotate deployment app-cache kubernetes.io/change-cause=\
"Image updated to 1.6.10"
```

```
deployment.apps/app-cache annotated
```

The rollout history now renders the change cause value for the current revision:

```
$ kubectl rollout history deployment app-cache
deployment.apps/app-cache
REVISION  CHANGE-CAUSE
1          <none>
2          Image updated to 1.6.10
```

Rolling Back to a Previous Revision

Problems can arise in production that require swift action. For example, the container image you just rolled out contains a crucial bug. Kubernetes gives you the option to roll back to one of the previous revisions in the rollout history. You can achieve this by using the `rollout undo` command. To pick a specific revision, provide the command-line option `--to-revision`. The command rolls back to the previous revision if you do not provide the option. Here, we are rolling back to revision 1:

```
$ kubectl rollout undo deployment app-cache --to-revision=1
deployment.apps/app-cache rolled back
```

As a result, Kubernetes performs a rolling update to all replicas with the revision 1.

ROLLBACKS AND PERSISTENT DATA

The `rollout undo` command does not restore any persistent data associated with applications. It only reverts the Deployment's Pod template (`.spec.template`) to a previous revision. Other Deployment settings, such as the replica count, remain unchanged.

The rollout history now lists revision 3. Given that we rolled back to revision 1, there's no more need to keep that entry as a duplicate. Kubernetes simply turns revision 1 into 3 and removes 1 from the list:

```
$ kubectl rollout history deployment app-cache
deployment.apps/app-cache
REVISION  CHANGE-CAUSE
```

Summary

The Deployment is an essential primitive for providing declarative updates and lifecycle management of Pods. The ReplicaSet performs the heavy lifting of managing those Pods, commonly referred to as replicas. A Deployment manages the ReplicaSet under the hood.

Deployments can easily roll out and roll back revisions of the application represented by an image running in the container. In this chapter you learned about the commands for controlling the revision history and its operations.

Exam Essentials

Know the ins and outs of a Deployment

Given that a Deployment is such a central primitive in Kubernetes, you can expect that the exam will test you on it. Know how to create and configure a Deployment.

Understand how the ReplicaSet supports replication

Learn how to scale to multiple replicas. One of the superior features of a ReplicaSet is its rollout functionality for new revisions. Practice how to roll out a new revision, inspect the rollout history, and roll back to a previous revision.

Differentiate the built-in deployment strategies

Learn how to configure the built-in strategies in the Deployment primitive and their options for fine-tuning the runtime behavior. You can implement even more sophisticated deployment scenarios with the help of the Deployment and Service primitives. Examples are the blue-green and canary deployment strategies, which require a multiphased rollout process, but will not be covered by the exam.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. A team member wrote a Deployment manifest but has trouble with creating the object from it. Help with finding the issue.
Navigate to the directory *app-a/ch11/misconfigured-deployment* of the checked-out GitHub repository [bmuschko/cka-study-guide](#).
Run a `kubectl` command to create the Deployment object defined in the file *fix-me-deployment.yaml*. Inspect the error message. Fix the Deployment manifest so that the object can be created.
2. Create a Deployment named `nginx` with three replicas. The Pods should use the `nginx:1.23.0` image and the name `nginx`. The Deployment uses the label `tier=backend`. The Pod template should use the label `app=v1`.
List the Deployment and ensure that the correct number of replicas is running.
Update the image to `nginx:1.23.4`.
Verify that the change has been rolled out to all replicas.
Assign the change cause “Pick up patch version” to the revision.
Have a look at the Deployment rollout history. Revert the Deployment to revision 1.
Ensure that the Pods use the image `nginx:1.23.0`.